Guia da P1 – Mandelbrot distribuído com MPI

Como funciona, o que cada arquivo faz e como rodar | José Levy e Diego Nascimento

Índice

1 O programa em uma frase **1**

2 Como tudo funciona **2**

2.1 Quem é quem . . . 2

2.2 O caminho de um `mpirun` . . . 2

2.3 Como as mensagens andam . . . 2

3 Os arquivos mais importantes **3**

3.1 `mandelbrot_mpi.c`: as definições . . . 3

3.2 `mandelbrot_mpi.c`: o `main` . . . 3

3.3 `mandelbrot_mpi.c`: o mestre . . . 4

3.4 `mandelbrot_mpi.c`: o escravo . . . 4

3.5 `mandelbrot_mpi.c`: calcular uma linha . . . 4

3.6 `mandelbrot_calc.c`: as fórmulas . . . 5

4 Passo a passo do zero: WSL (mestre) + VM Lubuntu **5**

4.1 Na VM: instalar e compilar (rede em NAT, com internet) . . . 5

4.2 No VirtualBox: rede em modo Bridge . . . 6

4.3 No Windows: WSL visível na rede (uma vez) . . . 6

4.4 No WSL: instalar e compilar . . . 6

4.5 No WSL: SSH sem senha até a VM . . . 7

4.6 No WSL: arquivo de máquinas e execução . . . 7

5 Adicionando mais máquinas (notebook do Diego, PC do lab) **7**

6 Problemas que já aconteceram e a solução **8**

7 Roteiro da apresentação (uns 5 minutos) **8**

8 Perguntas que o professor pode fazer **9**

1 O programa em uma frase

O mestre divide a imagem de 800×600 em 600 linhas, manda uma linha por vez para cada máquina escrava, recebe as linhas calculadas de volta, coloca cada uma no lugar certo da imagem e no final grava `saida.ppm`.

2 Como tudo funciona

2.1 Quem é quem

Cada cópia do programa é um **processo**, identificado por um número (*rank*). O rank 0 é o mestre; os outros são escravos. O mestre não calcula, só distribui e junta; por isso o computador do mestre também roda um escravo, para não ficar com a CPU parada.

Máquina	**Processos**	**Papel**
WSL do José (`Pesquisa`), 192.168.0.4	2	rank 0 = mestre, rank 1 = escravo
VM Lubuntu (`jose-virtualbox`), 192.168.0.5	2	ranks 2 e 3 = escravos

Isso vem do arquivo de máquinas (`~/hosts_vm`): `localhost slots=2` e `joselrds@192.168.0.5 slots=2`. Com o notebook do Diego ou o PC do lab, basta acrescentar uma linha por máquina (Seção 5).

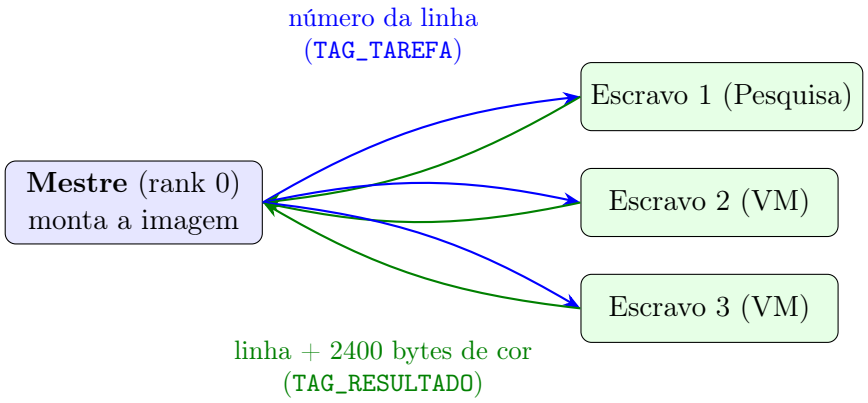
2.2 O caminho de um `mpirun`

1. **Início, por SSH.** O `mpirun` lê o arquivo de máquinas e, para cada máquina remota, roda sozinho algo como `ssh joselrds@192.168.0.5 orted ...`. O `orted` é o "ajudante" do Open MPI: ele sobe na outra máquina e abre lá os processos do programa. Por isso o mestre precisa entrar nas outras máquinas por SSH **sem senha**.

2. **Execução, por TCP direto.** Depois disso o SSH não é mais usado: os processos trocam mensagens (`MPI_Send`/`MPI_Recv`) por conexões TCP entre os IPs das máquinas. Os escravos abrem conexões **de volta** para o mestre, e por isso o firewall do Windows precisa liberar a entrada no WSL.

3. **Fim.** O mestre grava `saida.ppm` na pasta de onde o `mpirun` foi chamado.

2.3 Como as mensagens andam



1. Cada escravo começa mandando o **nome do computador** dele (`TAG_NOME`). É isso que faz aparecer "Maquina 2 (jose-virtualbox)" na tela: a prova de que outro computador está calculando.

2. O mestre manda a linha 0 para o escravo 1, a linha 1 para o escravo 2, e assim por diante.

3. Cada vez que um escravo devolve uma linha, o mestre guarda a linha na imagem e **manda a próxima linha que falta para esse mesmo escravo**.

4. Quando acabam as 600 linhas, o mestre recebe as últimas e manda `END` (-1) para cada escravo, que encerra.

5. O mestre grava `saida.ppm`.

Por que linha por linha (distribuição dinâmica) e não 200 linhas fixas para cada um? No Mandelbrot algumas linhas demoram muito mais que outras (a parte cinza roda as 20000 iterações em cada pixel). Com divisão fixa, uma máquina ficaria sobrecarregada e as outras paradas. Com a distribuição dinâmica, quem termina antes pega mais linhas. Na execução com a VM ficou: Pesquisa 363 linhas, VM 120 + 117. O escravo do Pesquisa pega mais porque está no mesmo computador do mestre e as respostas dele não passam pela rede.

3 Os arquivos mais importantes

Arquivo	O que tem
`codigo_fonte/mandelbrot_mpi.c`	**O programa distribuído**: mestre e escravo. É a entrega principal.
`codigo_fonte/mandelbrot_calc.c/.h`	As fórmulas do fractal e da cor, copiadas do código do professor para C.
`hosts` (ou `~/hosts_vm`)	Quais máquinas participam e quantos processos cada uma roda.
`codigo_fonte/Makefile`	`make mpi` faz o mesmo que o `mpicc` da Seção 4.
`mestre_escravo_exemplo.c`	Exemplo do professor que serviu de base (sem alteração).
`mandelbrot.cpp`, `Mandel.h`, `Palette.*`, `Color.h`, `X11Display.*`	Mandelbrot serial do professor, com janela X11 (sem alteração).

O zip `codigo_P1_mandelbrot_mpi.zip` tem exatamente esses arquivos, idênticos aos do GitHub.

3.1 `mandelbrot_mpi.c`: as definições

```c
#define IMG_WIDTH   800
#define IMG_HEIGHT  600
#define MAXITER     20000
#define TAG_TAREFA     110   /* mestre -> escravo: numero da linha     */
#define TAG_RESULTADO  111   /* escravo -> mestre: a linha calculada   */
#define TAG_NOME       112   /* escravo -> mestre: nome do computador  */
#define END            -1    /* "acabou, pode encerrar"                */

typedef struct {
    int linha;                          /* qual linha e' esta        */
    unsigned char pixels[IMG_WIDTH * 3];/* 800 pixels x (R, G, B)   */
} LinhaResultado;
```

As *tags* são etiquetas das mensagens: quem recebe pede uma tag específica e só recebe mensagens daquele tipo. O resultado viaja como uma `struct` inteira, numa mensagem só, com o número da linha junto. Por isso a ordem em que as linhas chegam não importa.

3.2 `mandelbrot_mpi.c`: o `main`

O mesmo executável roda em todas as máquinas; o `main` só olha o *rank* para decidir o papel: rank 0 chama `mestre()`, os outros chamam `escravo()` (igual ao exemplo do professor). Ele começa com `MPI_Init` e termina com `MPI_Finalize`.

3.3 `mandelbrot_mpi.c`: o mestre

Primeiro manda uma linha para cada escravo; depois fica neste laço, que é o coração do programa:

```
while (tarefa < IMG_HEIGHT) {
    MPI_Recv(&resultado, sizeof(LinhaResultado), MPI_BYTE, MPI_ANY_SOURCE,
             TAG_RESULTADO, MPI_COMM_WORLD, &stat);
    memcpy(&imagem[(size_t) resultado.linha * IMG_WIDTH * 3],
           resultado.pixels, IMG_WIDTH * 3);
    MPI_Send(&tarefa, 1, MPI_INT, stat.MPI_SOURCE, TAG_TAREFA, MPI_COMM_WORLD);
    tarefa++;
}
```

- `MPI_ANY_SOURCE`: aceita a resposta de **quem terminar primeiro**, sem esperar um escravo específico.
- `stat.MPI_SOURCE`: diz quem mandou; é para esse escravo que vai a próxima linha.
- `memcpy`: aqui a figura é **integrada**. A linha y começa na posição $y \times 800 \times 3$ do vetor da imagem.

Depois do laço sobra um resultado pendente por escravo: o mestre recebe cada um e responde com `END`. No fim, `salvar_ppm` grava o vetor inteiro:

```
fprintf(f, "P6\n%d %d\n255\n", largura, altura);   /* cabecalho do PPM */
fwrite(imagem, 1, (size_t) largura * altura * 3, f); /* os pixels RGB    */
```

3.4 `mandelbrot_mpi.c`: o escravo

```
MPI_Get_processor_name(nome, &tam_nome);
MPI_Send(nome, MPI_MAX_PROCESSOR_NAME, MPI_CHAR, 0, TAG_NOME, MPI_COMM_WORLD);

while (1) {
    MPI_Recv(&tarefa, 1, MPI_INT, 0, TAG_TAREFA, MPI_COMM_WORLD, &stat);
    if (tarefa == END) {
        break;
    }
    printf("Maquina %d (%s): calculando linha %d\n", rank, nome, tarefa);
    calcula_linha(tarefa, resultado.pixels);
    resultado.linha = tarefa;
    MPI_Send(&resultado, sizeof(LinhaResultado), MPI_BYTE, 0, TAG_RESULTADO, MPI_COMM_WORLD)
    ;
}
```

O escravo se apresenta (manda o nome do computador) e depois repete: recebe um número de linha, calcula, devolve. Se a variável `APELIDO` estiver definida (`mpirun -x APELIDO=diego ...`), ele usa o apelido no lugar do nome.

3.5 `mandelbrot_mpi.c`: calcular uma linha

```
static void calcula_linha(int y, unsigned char *buf_linha)
{
    double dx = (ZMAX_R - ZMIN_R) / (double) IMG_WIDTH;
    double dy = (ZMAX_I - ZMIN_I) / (double) IMG_HEIGHT;
    for (x = 0; x < IMG_WIDTH; x++) {
        double cr = ZMIN_R + (double) x * dx;   /* pixel (x, y) vira o  */
        double ci = ZMIN_I + (double) y * dy;   /* ponto c = cr + ci*i  */
        int iter = calc_point(cr, ci, MAXITER);
```

```
        get_color(iter, MAXITER, &buf_linha[x*3], &buf_linha[x*3+1], &buf_linha[x*3+2]);
    }
}
```

É o mesmo mapeamento do `Mandel::calcAndShow` do professor ($c = z_{min} + (x \cdot dx,\ y \cdot dy)$), só que para uma linha y fixa, e com a mesma região com zoom.

3.6 `mandelbrot_calc.c`: as fórmulas

```
int calc_point(double cr, double ci, int maxiter)
{
    double zr = 0.0, zi = 0.0, novo_zr, novo_zi, m2;
    int iter = 0;
    do {
        novo_zr = zr * zr - zi * zi + cr;   /* (a+bi)^2 = (a^2-b^2) + 2abi */
        novo_zi = 2.0 * zr * zi + ci;
        zr = novo_zr;
        zi = novo_zi;
        m2 = zr * zr + zi * zi;             /* |z|^2 */
        iter++;
    } while (m2 < 4.0 && iter < maxiter);
    return iter;
}
```

É a iteração $z_{n+1} = z_n^2 + c$ do `Mandel::calcPoint`, sem `std::complex`: a parte real e a imaginária ficam em duas variáveis `double`. O número de iterações vira cor em `get_color`, que é a mesma senoide do `RGBPalette::getColor` com os mesmos parâmetros (`0.4, 0.95, 0.65, 0.45, 0.5, 0.5, 0.5`):

```
double n  = (double) iter / (double) maxiter;
double ne = pow(n, PALETTE_EXP);
double fr = pow(sin(ne * 5 * 3.14 * PALETTE_FREQ_R + PALETTE_FASE_R * 10 * 3.14 / 2), 2);
/* fg e fb iguais, com as frequencias de G e B; depois de 0..1 para 0..255 */
```

Reescrevemos só essas duas funções em C porque o exemplo de MPI do professor é em C (`mpicc`) e o serial é C++ com janela X11: assim as máquinas escravas não precisam de interface gráfica.

4 Passo a passo do zero: WSL (mestre) + VM Lubuntu

Todos os comandos são digitados à mão; nenhum script é usado. Os exemplos usam os nossos dados: usuário `joselrds`, IP da VM 192.168.0.5, rede 192.168.0.x.

4.1 Na VM: instalar e compilar (rede em NAT, com internet)

Na instalação do Lubuntu, "Apagar disco" só apaga o disco virtual da VM.

```
sudo apt update
sudo apt install -y git build-essential openmpi-bin libopenmpi-dev openssh-server
sudo systemctl enable --now ssh          # SSH ligado sempre que a VM iniciar
cd ~
git clone https://github.com/levyrodrigues23/mpi-mandelbrot-p1.git
cd ~/mpi-mandelbrot-p1/codigo_fonte
mpicc -O2 -o mandelbrot_mpi mandelbrot_mpi.c mandelbrot_calc.c -lm
mpirun --oversubscribe -np 4 ./mandelbrot_mpi   # teste local: gera saida.ppm
which mpirun                                    # tem que ser /usr/bin/mpirun
whoami
sudo poweroff
```

`mpicc` é o `gcc` com o MPI; `-O2` (letra O) liga as otimizações, `-o mandelbrot_mpi` dá o nome do executável (sem `.c`) e `-lm` liga a biblioteca de matemática. O projeto tem que ficar em `~/mpi-mandelbrot-p1`, o mesmo caminho do mestre. Se ele estiver em outra pasta, um atalho resolve (no nosso caso, a VM tinha o projeto em `~/Documents/prova`):

```
ln -s ~/Documents/prova/mpi-mandelbrot-p1 ~/mpi-mandelbrot-p1
```

4.2 No VirtualBox: rede em modo Bridge

1. No PowerShell, `Get-NetAdapter` mostra qual placa está com **Status Up** (no nosso caso, o adaptador Ethernet USB; o Wi-Fi estava desconectado).
2. Configurações da VM → Rede → **Placa em modo Bridge** → Nome: essa placa. Em Avançado, **Cabo conectado** marcado.
3. Ligar a VM e conferir com `ip -4 addr`: tem que aparecer um IP da mesma faixa do Windows (ex.: 192.168.0.5). Se aparecer só `127.0.0.1`, a placa escolhida está errada.

4.3 No Windows: WSL visível na rede (uma vez)

Arquivo `C:\Users\josel\.wslconfig`:

```
[wsl2]
networkingMode=mirrored
```

No PowerShell como administrador:

```
Set-NetFirewallHyperVVMSetting -Name '{40E0AC32-46A5-438A-A0B2-2B479E8F2E90}' -
    DefaultInboundAction Allow
wsl --shutdown
```

O modo espelhado faz o WSL usar o mesmo IP do Windows; o segundo comando libera as conexões de entrada no WSL. O `ping` da VM para o WSL pode falhar (o Windows bloqueia ping), mas o que importa é o TCP: `ssh josel@192.168.0.4` a partir da VM tem que pedir senha.

4.4 No WSL: instalar e compilar

```
sudo apt update
sudo apt install -y git build-essential openmpi-bin libopenmpi-dev openssh-server
    imagemagick
sudo service ssh start
which mpirun mpicc orted     # os tres em /usr/bin: o MESMO Open MPI da VM
hostname -I                  # mesma faixa da VM, ex.: 192.168.0.4
grep HWLOC ~/.bashrc || sed -i '1i export HWLOC_COMPONENTS=-gl' ~/.bashrc
cd ~
git clone https://github.com/levyrodrigues23/mpi-mandelbrot-p1.git
cd ~/mpi-mandelbrot-p1/codigo_fonte
mpicc -O2 -o mandelbrot_mpi mandelbrot_mpi.c mandelbrot_calc.c -lm
```

A linha do `HWLOC_COMPONENTS` evita que o `mpirun` trave no WSL (o `hwloc` tenta falar com um servidor gráfico que não responde). Ela vai na **primeira linha** do `~/.bashrc` para valer também nas sessões SSH. Depois de mexer no `.bashrc`, feche a janela do Ubuntu e abra outra.

4.5 No WSL: SSH sem senha até a VM

```
cd ~
ls ~/.ssh/id_*.pub || ssh-keygen      # so cria a chave se nao existir
ssh-copy-id joselrds@192.168.0.5      # pede a senha da VM uma unica vez
ssh joselrds@192.168.0.5 hostname     # mostra o nome da VM SEM pedir senha
```

O último comando só mostra o nome e volta. Se você entrar na VM com `ssh` (o prompt vira `joselrds@jose-virtualbox`), digite `exit` antes de continuar: o `mpirun` roda **no WSL**.

4.6 No WSL: arquivo de máquinas e execução

```
cd ~
nano hosts_vm
# conteudo:
localhost slots=2
joselrds@192.168.0.5 slots=2
```

Com o prompt `josel@Pesquisa:~$` (pasta pessoal):

```
mpirun --hostfile hosts_vm -np 4 \
  --mca btl_tcp_if_include 192.168.0.0/24 \
  --mca oob_tcp_if_include 192.168.0.0/24 \
  hostname                                # 2x Pesquisa e 2x jose-virtualbox

mpirun --hostfile hosts_vm -np 4 \
  --mca btl_tcp_if_include 192.168.0.0/24 \
  --mca oob_tcp_if_include 192.168.0.0/24 \
  mpi-mandelbrot-p1/codigo_fonte/mandelbrot_mpi

convert saida.ppm saida.png && explorer.exe saida.png
```

O que cada parte faz:

- `--hostfile hosts_vm` e `-np 4`: as máquinas do arquivo e o total de processos (a soma dos `slots`).
- `oob_tcp_if_include`: a rede que o `mpirun` usa para controlar os processos. `btl_tcp_if_include`: a rede das mensagens do programa. `192.168.0.0/24` quer dizer "todos os IPs que começam com 192.168.0" (os três primeiros números dos IPs + `.0/24`). Sem isso o Open MPI tenta as placas virtuais do WSL e trava.
- `hostname` no lugar do programa: teste rápido que mostra em qual computador cada processo abriu.
- O caminho do programa é relativo à pasta pessoal. Na VM, onde `/home/josel` não existe, o processo começa na pasta pessoal de lá (`/home/joselrds`), e o mesmo caminho funciona.

5 Adicionando mais máquinas (notebook do Diego, PC do lab)

As máquinas têm que estar no **mesmo roteador** (IP começando com 192.168.0.). Em cada máquina nova: os mesmos passos do WSL (Seção 4), incluindo o `.wslconfig` e o firewall se for WSL. Antes, conferir que não há um Open MPI antigo no `PATH`:

```
grep -n ompi-igual ~/.bashrc ~/.profile               # se aparecer algo:
sed -i '/ompi-igual/s/^/# /' ~/.bashrc ~/.profile     # e abrir um terminal novo
```

No WSL do José:

```
ssh-copy-id USUARIO@IP_DA_MAQUINA
ssh USUARIO@IP_DA_MAQUINA 'hostname; which mpirun orted; echo $HWLOC_COMPONENTS'
# tem que responder sem senha: o nome, /usr/bin/mpirun, /usr/bin/orted e -gl
```

E uma linha nova no `hosts_vm` (ex.: `USUARIO@IP_DA_MAQUINA slots=2`), trocando o `-np` pela nova soma dos slots (6, no exemplo).

6 Problemas que já aconteceram e a solução

O que aparece	**Causa e solução**
VM só com `127.0.0.1`	Bridge na placa errada: usar a que está *Up* no `Get-NetAdapter`.
`git clone`: *invalid username or token*	O repositório é privado e o GitHub não aceita a senha da conta: usar um token (`ghp_...`) ou deixar o repositório público.
undefined reference to main	Faltou o `mandelbrot_mpi.c` no `mpicc` (nome com `_`, não espaço).
`Data unpack had inadequate space`	Open MPI diferente entre as máquinas. No José, `which mpirun` apontava para `~/ompi-igual` (compilado à mão). Comentar as linhas no `.bashrc`, abrir terminal novo e recompilar.
multiple definitions of the slot count	O `mpirun` foi rodado de dentro da VM: `exit` e rodar no WSL.
Could not resolve hostname locahost	Erro de digitação no arquivo de máquinas (`localhost`).
unable to open the hostfile	O `mpirun` não foi chamado da pasta onde está o arquivo (`cd ~`).
could not access or execute an executable	O programa não está em `~/mpi-mandelbrot-p1/codigo_fonte` na outra máquina (usar o atalho `ln -s`).
`mpirun` parado sem mostrar nada	Falta o `HWLOC_COMPONENTS=-gl` ou as opções `--mca ..._if_include`.

7 Roteiro da apresentação (uns 5 minutos)

1. **O problema** (30 s): o Mandelbrot serial do professor calcula pixel por pixel num processo só. Cada pixel é independente, então dá pra dividir.
2. **A ideia** (1 min): usamos o exemplo mestre/escravo do professor; a tarefa virou "calcular uma linha". Mostrar o diagrama da Seção 2.3.
3. **Por que linha por linha** (30 s): linhas têm custos muito diferentes; com distribuição dinâmica ninguém fica parado.
4. **Código** (1 min 30 s): abrir `mandelbrot_mpi.c` e mostrar o laço do mestre e o do escravo.
5. **Rodar ao vivo** (1 min): o `mpirun` com `hostname` e depois com o programa. Apontar os nomes de computadores diferentes nas mensagens.
6. **Mostrar a imagem** (30 s): `convert saida.ppm saida.png` e abrir.

Plano B: se a rede da sala não deixar as máquinas se verem, usar o hotspot do celular. Se nada funcionar, `mpirun --oversubscribe -np 4 ./mandelbrot_mpi` dentro de `codigo_fonte` mostra o programa numa máquina só, e o relatório tem os prints das execuções em rede.

8 Perguntas que o professor pode fazer

Onde cada máquina faz uma parte da imagem?
Cada escravo calcula as linhas que recebe. Na saída aparece "Maquina 2 (jose-virtualbox): calculando linha 7": quem calculou cada linha.

Onde o mestre integra a figura?
No `memcpy` do laço do mestre: cada linha recebida é copiada para a posição `linha * 800 * 3` do vetor da imagem. No final, `salvar_ppm` grava o vetor inteiro.

Como o `mpirun` abre o programa na outra máquina?
Por SSH (sem senha, com a chave copiada pelo `ssh-copy-id`): ele inicia o `orted` lá, que abre os processos. Depois as mensagens vão por TCP direto, sem SSH.

Por que o mestre não calcula também?
Seguimos o modelo do exemplo do professor, em que o mestre só coordena. Para não desperdiçar o computador do mestre, ele também roda um escravo (`slots=2`).

Por que cada máquina não calcula o mesmo número de linhas?
A distribuição é dinâmica: quem termina antes pega a próxima linha. Com a VM ficou 363 / 120 / 117.

Por que reescreveram o cálculo em C?
O exemplo de MPI é em C e o serial é C++ com janela X11. Passamos só `calcPoint` e `getColor` para C, com os mesmos parâmetros; os arquivos do professor não foram alterados.

O que é o arquivo de máquinas e o `slots`?
A lista das máquinas que o `mpirun` usa e quantos processos abre em cada uma.

Que problema deu mais trabalho?
Fazer as máquinas conversarem: WSL visível na rede, SSH sem senha, o `hwloc` travando o `mpirun` no WSL e a exigência de usar a mesma instalação do Open MPI em todas as máquinas (`Data unpack had inadequate space`).

Como sabem que a imagem está certa?
Mesma região, mesmas fórmulas e mesma paleta do serial. E, como está no relatório, a imagem da execução com o notebook do Diego e a da execução com o lab-ubuntu são idênticas byte a byte (`md5sum`).